claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_34138e24-10c\agent-a77015e15bfc5f6b0.jsonl`
- SHA-256: `3064967349715a10e3504dd894174703616304a9ea10039d3ba0027eff456741`
- Source modified: `2026-06-18T01:14:02+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_34138e24-10c`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:10:52.226Z)

```
You are reviewing a small bug-fix commit in a Python repo (badminton-highlight-indexer).
Work in this git worktree (read-only ? DO NOT modify, commit, or push anything):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
First run: `git -C "C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/ablation-schema-fix" diff origin/master..HEAD` to see the exact
change. Then read the
surrounding code in backend/eval/ablation.py (load_videos, _is_fusion_schema, litmus_config,
default_signals) and tests/test_ablation.py (the litmus tests, _golden_present, the new
regression tests, _write_learned_corpus).

CONTEXT: The fix makes ablation.load_videos() skip "windowing-only" golden_features.json files
(leaner schema: start/end + velocity stats, NO shuttle/person/audio cue dicts) so a MIXED
corpus (6 full-fusion files + 5 leaner files) loads only the 6 ablatable files instead of
crashing with KeyError: 'shuttle'. The litmus must still reproduce the recorded Gate-A numbers
(?F1?0.074, 6/6 wins, p?0.031) on exactly those 6 files.

Be adversarial: actively try to find a reason this should NOT be merged as-is. Report concrete,
verifiable findings only (cite file:line). Do not invent issues. For each finding give a
severity: must-fix (a real bug / regression / breaks the litmus), should-fix, nit, or none.
Set blocks_merge=true ONLY if you found a genuine must-fix. Return the structured object.

YOUR LENS ? test quality & regression faithfulness:
- Would the two NEW tests actually FAIL against the OLD (pre-fix) load_videos? (i.e. do they pin
the bug, not just pass trivially?)
- Are the assertions meaningful (names of loaded videos, single gate_a row, num_videos==6) or
could they pass for the wrong reason?
- Does _write_windowing_only mirror the REAL lean schema keys? Does the litmus run in the test
exercise the same code path that crashed?
- Is the new private-name access (ablation._is_fusion_schema) reasonable for a test?
- Any missing case worth a test (e.g. a file with person but no shuttle)?
```

2. assistant (2026-06-18T01:10:54.103Z)

```
I'll start by examining the exact change and the surrounding code.
```

3. user (2026-06-18T01:10:55.760Z)

```
diff --git a/backend/eval/ablation.py b/backend/eval/ablation.py
index fec3e7d..968eefd 100644
--- a/backend/eval/ablation.py
+++ b/backend/eval/ablation.py
@@ -383,16 +383,37 @@ def litmus_config() -> AblationConfig:
 # ---------------------------------------------------------------------------- corpus I/O
```

```
+def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+    """True iff every candidate row carries the per-cue feature dicts the ablation scorer reads
+    (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
+
+    The velocity-windowing / distill path persists a LEANER ``*_golden_features.json`` row
+    (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through fusion
+    feature extraction. ``load_videos`` SKIPS those rather than crash on ``c["shuttle"]`` ? so
a
+    mixed corpus (some fusion files, some windowing-only) loads the ablatable subset cleanly.
+    An empty candidate list is not ablatable either."""
+    return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
+
 def load_videos(features_dir: str) -> List[VideoCandidates]:
-    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
-    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
-    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
-    mirroring ``experiment._feature_column``'s tolerance."""
+    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
+    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
+    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
+    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
+
+    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
+    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
+    ablatable subset cleanly (see ``_is_fusion_schema``)."""
     videos: List[VideoCandidates] = []
     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
         with open(path, encoding="utf-8") as f:
             d = json.load(f)
         cands = d["candidates"]
+        if not _is_fusion_schema(cands):
+            logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
+                        "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
+                        os.path.basename(path))
+            continue
         intervals = [(float(c["start"]), float(c["end"])) for c in cands]
         features: Dict[str, List[float]] = {}
         for fn in SHUTTLE_COLUMNS:
diff --git a/tests/test_ablation.py b/tests/test_ablation.py
index b29ecbb..224d497 100644
--- a/tests/test_ablation.py
+++ b/tests/test_ablation.py
@@ -357,6 +357,48 @@ def test_load_videos_tolerates_missing_audio(tmp_path):
     assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not crashed


+def _write_windowing_only(d: str, name: str) -> None:
+    """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the
+    R3/R4 corpus-growth path persists for clips that never went through fusion feature
+    extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts). The
+    ablation loader must SKIP these, never crash on ``c["shuttle"]``."""
+    cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
+              "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
+    data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
+            "candidates": cands, "gts": [[0.0, 5.0]]}
+    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
+        json.dump(data, f)
+
+
+def test_is_fusion_schema_distinguishes_lean_and_full():
+    """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and rejects
+    the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
ablatable."""
+    full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
```

```
+    no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}]          # audio
optional
+    lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
+    assert ablation._is_fusion_schema(full) is True
+    assert ablation._is_fusion_schema(no_audio) is True
+    assert ablation._is_fusion_schema(lean) is False
+    assert ablation._is_fusion_schema([]) is False
+
+
+def test_load_videos_skips_windowing_only_schema(tmp_path):
+    """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files load; the
+    leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than crashed
on,
+    and the litmus path runs end-to-end over the filtered ablatable subset."""
+    d = str(tmp_path)
+    _write_learned_corpus(d, n=6)                                  # 6 full-fusion files
(v0..v5)
+    for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"):  # 3 leaner real-clip names
+        _write_windowing_only(d, name)
+    # only the fusion-schema files load ? the leaner ones never crash the loader
+    videos = ablation.load_videos(d)
+    assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
+    # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed corpus
+    report = run_ablation(videos, ablation.litmus_config())
+    assert [r.signal for r in report.rows] == ["gate_a"]
+    assert report.num_videos == 6
+
+
 # ------------------------------------------------------------------------------ the golden
litmus

 GOLDEN_DIR = os.path.join(
@@ -364,7 +406,20 @@ GOLDEN_DIR = os.path.join(


 def _golden_present() -> bool:
-    return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
+    """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
cue
+    columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
+    ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus (the n=11
+    mixed-schema bug)."""
+    n = 0
+    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
+        try:
+            with open(p, encoding="utf-8") as f:
+                cands = json.load(f).get("candidates") or []
+        except (OSError, ValueError):
+            continue
+        if ablation._is_fusion_schema(cands):
+            n += 1
+    return n >= 6


 @pytest.mark.skipif(not _golden_present(),
```

4. assistant (2026-06-18T01:10:58.014Z)

Now let me read the surrounding code in both files.

5. user (2026-06-18T01:11:00.140Z)

```
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
```

```
382
383     # ---------------------------------------------------------------------------- corpus
I/O
384
385
386     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
387         """True iff every candidate row carries the per-cue feature dicts the ablation
scorer reads
388         (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
389
390         The velocity-windowing / distill path persists a LEANER ``*_golden_features.json``
row
391         (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through
fusion
392         feature extraction. ``load_videos`` SKIPS those rather than crash on
``c["shuttle"]`` ? so a
393         mixed corpus (some fusion files, some windowing-only) loads the ablatable subset
cleanly.
394         An empty candidate list is not ablatable either."""
395         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
396
397
398     def load_videos(features_dir: str) -> List[VideoCandidates]:
399         """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a
``VideoCandidates``
400         (intervals + shuttle + person + audio columns + golden gts). Audio columns are
optional
401         (older JSONs predate the audio augment) ? a missing block defaults those columns to
0.0 so
402         the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
403
404         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
405         R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus
loads the
406         ablatable subset cleanly (see ``_is_fusion_schema``)."""
407         videos: List[VideoCandidates] = []
408         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
409             with open(path, encoding="utf-8") as f:
410                 d = json.load(f)
411             cands = d["candidates"]
412             if not _is_fusion_schema(cands):
413                 logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
414                             "(no shuttle/person cue dicts), not an ablatable fusion-feature
file",
415                             os.path.basename(path))
416                 continue
417             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
418             features: Dict[str, List[float]] = {}
419             for fn in SHUTTLE_COLUMNS:
420                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
421             for fn in PERSON_COLUMNS:
422                 features[fn] = [float(c["person"][fn]) for c in cands]
423             for fn in AUDIO_COLUMNS:
424                 features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
425             gts = [(float(a), float(b)) for a, b in d["gts"]]
426             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
427                                           features=features, gts=gts))
428         return videos
429
430
431     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
432         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
```

```
433         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
434         (distinct from the static Gate-A litmus)."""
435         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
436         return experiment.Variant(name="full_learned", feature_names=cols,
437                                   threshold=threshold, min_crossings=min_crossings)
438
439
440     # ------------------------------------------------------------------------ table
rendering
441
442
443     def format_table(report: AblationReport) -> str:
444         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
445         rows = report.sorted_rows()
446         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
447                   f"  ·  n={report.num_videos}  ·  IoU {report.iou}"
448                   f"  ·  {'LOVO-honest' if report.honest_lovo else 'static'}"
449                   f"  ·  {report.granularity.upper()}")
450         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
451         widths = [18, 17, 18, 14, 5, 22]
452
453         def _fmt_row(cells: Sequence[str]) -> str:
454             return "  ".join(c.ljust(w) for c, w in zip(cells, widths))
455
456         lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
457         for r in rows:
458             ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
459             sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
460             verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
461             lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
462                                    "yes" if r.ci_excludes_zero else "no", verdict]))
463         return "\n".join(lines)
464
465
466     # ------------------------------------------------------------------------ PDF
export
467
468
469     def export_pdf(report: AblationReport, path: str) -> str:
```

6. user (2026-06-18T01:11:00.447Z)

```
1       """GPU-free tests for the LOSO/LOGO ablation runner (``backend.eval.ablation``).
2
3       Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5         - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise
signal is
6           *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns /
zeroes
7           the gate knob without mutating ``full``.
8         - the structural-guard exemption ? the SAME signal marked ``structural=True`` is
reported
9           (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no
lift".
10        - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
stronger
11          than the stats support (the no-over-claim contract).
12        - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
13        - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
14          reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
```

```
15          cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16      """
17      from __future__ import annotations
18
19      import glob
20      import json
21      import os
22
23      import pytest
24
25      from backend.eval import ablation
26      from backend.eval.ablation import (
27          AblationConfig,
28          AblationReport,
29          Signal,
30          ablate_one,
31          run_ablation,
32      )
33      from backend.eval.experiment import Variant, VideoCandidates
34
35
36      # ------------------------------------------------------------------------ synthetic
fixtures
37
38
39      def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
40          """``n`` videos with a ``net_crossings`` gate signal.
41
42          ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
``min_crossings=2`` gate
43          perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
44          has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
45          videos: list[VideoCandidates] = []
46          for k in range(n):
47              intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
48              junk_val = 5.0 if not separating else 0.0
49              feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
50              gts = [(0.0, 10.0), (20.0, 30.0)]
51              videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
52          return videos
53
54
55      def _learned_videos(n: int = 6) -> list[VideoCandidates]:
56          """``n`` videos where a PERSON feature (``players_in_court``) cleanly separates
rallies from
57          junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
58          ablation must show person as the load-bearing group and the others as washes ? a
deterministic
59          LOGO/LOSO correctness fixture (mirrors ``test_fusion_compare``'s construction)."""
60          videos: list[VideoCandidates] = []
```

7. assistant (2026-06-18T01:11:04.454Z)

Let me look at the test helpers `_write_learned_corpus`, `_golden_present`, and the litmus test, plus the `litmus_config` / `default_signals` referenced.

8. user (2026-06-18T01:11:07.310Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix/backend/eval/ablation.py:65:SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:67:PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court",
"two_sided_motion", "mean_player_motion",
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:69:AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count",
"audio_hit_rate", "audio_hit_strength_mean")
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:352:def default_signals() -> List[Signal]:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:363:        Signal("person", "person", PERSON_COLUMNS),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:364:        Signal("audio", "audio", AUDIO_COLUMNS),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:368:def static_gate_a_signal() -> Signal:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:375:def litmus_config() -> AblationConfig:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:380:    return AblationConfig(full=full,
signals=[static_gate_a_signal()], granularity="signal")
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:419:        for fn in SHUTTLE_COLUMNS:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:421:        for fn in PERSON_COLUMNS:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:423:        for fn in AUDIO_COLUMNS:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:435:    cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) +
list(AUDIO_COLUMNS)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/backend/eval/ablation.py:616:        cfg = AblationConfig(full=learned_full_variant(),
signals=default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:64:        for fn in ablation.SHUTTLE_COLUMNS:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:66:        for fn in ablation.PERSON_COLUMNS:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:70:        for fn in ablation.AUDIO_COLUMNS:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:134:    groups =
ablation._collapse_to_groups(ablation.default_signals())
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:149:    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:166:
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:169:
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:171:    assert len(loso.rows) == len(ablation.default_signals())
# 5 LOSO rows
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:178:    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:185:    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals())
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:228:    sig = next(s for s in ablation.default_signals() if s.group
== "gate_a")
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:240:    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
```

```
fix/tests/test_ablation.py:283:
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:296:    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:310:
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:328:
signals=ablation.default_signals(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:348:            "shuttle": {fn: 1.0 for fn in
ablation.SHUTTLE_COLUMNS},
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:349:            "person": {fn: 0.0 for fn in
ablation.PERSON_COLUMNS},
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:390:    _write_learned_corpus(d, n=6)
# 6 full-fusion files (v0..v5)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:408:def _golden_present() -> bool:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:425:@pytest.mark.skipif(not _golden_present(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:453:@pytest.mark.skipif(not _golden_present(),
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:487:def _write_learned_corpus(d: str, n: int = 6) -> None:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:491:            shuttle = {fn: 1.0 for fn in
ablation.SHUTTLE_COLUMNS}
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:492:            person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:495:            audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:510:    _write_learned_corpus(d)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:528:    _write_learned_corpus(d)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-
fix/tests/test_ablation.py:539:    _write_learned_corpus(d, n=1)
```

9. assistant (2026-06-18T01:11:13.100Z)

Now let me read the key sections: `litmus_config`/`default_signals`/`static_gate_a_signal` in ablation.py, the litmus test (lines 425-545), and `_write_learned_corpus`.

10. user (2026-06-18T01:11:14.061Z)

```
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
```

```
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
```

11. user (2026-06-18T01:11:15.654Z)

```
340
341     # ------------------------------------------------------------------------ corpus loader
(no GPU)
342
343
344     def test_load_videos_tolerates_missing_audio(tmp_path):
345         """An older feature JSON without an ``audio`` block still loads ? audio columns
default to
346         0.0 (loader never crashes on a pre-audio JSON)."""
347         cand = {"start": 0.0, "end": 10.0,
348                 "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
349                 "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
350                 "label": 1}
351         data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
352                 "candidates": [cand], "gts": [[0.0, 10.0]]}
353         with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
354             json.dump(data, f)
355         videos = ablation.load_videos(str(tmp_path))
356         assert len(videos) == 1
357         assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not
crashed
358
359
360     def _write_windowing_only(d: str, name: str) -> None:
361         """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema
the
362         R3/R4 corpus-growth path persists for clips that never went through fusion feature
363         extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts).
The
364         ablation loader must SKIP these, never crash on ``c["shuttle"]``."""
365         cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
366                   "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
367         data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
368                 "candidates": cands, "gts": [[0.0, 5.0]]}
369         with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
```

```
f:
370             json.dump(data, f)
371
372
373     def test_is_fusion_schema_distinguishes_lean_and_full():
374         """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and
rejects
375         the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
ablatable."""
376         full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
377         no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}]          #
audio optional
378         lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
379         assert ablation._is_fusion_schema(full) is True
380         assert ablation._is_fusion_schema(no_audio) is True
381         assert ablation._is_fusion_schema(lean) is False
382         assert ablation._is_fusion_schema([]) is False
383
384
385     def test_load_videos_skips_windowing_only_schema(tmp_path):
386         """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files
load; the
387         leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than
crashed on,
388         and the litmus path runs end-to-end over the filtered ablatable subset."""
389         d = str(tmp_path)
390         _write_learned_corpus(d, n=6)                                  # 6 full-fusion files
(v0..v5)
391         for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"):  # 3 leaner real-clip
names
392             _write_windowing_only(d, name)
393         # only the fusion-schema files load ? the leaner ones never crash the loader
394         videos = ablation.load_videos(d)
395         assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
396         # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed
corpus
397         report = run_ablation(videos, ablation.litmus_config())
398         assert [r.signal for r in report.rows] == ["gate_a"]
399         assert report.num_videos == 6
```

12. user (2026-06-18T01:11:16.117Z)

```
425     @pytest.mark.skipif(not _golden_present(),
426                         reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
427     def test_litmus_reproduces_gate_a():
428         """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
429         result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
430         This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
431         where the golden substrate is absent; runs on the owner box where it is present."""
432         videos = ablation.load_videos(GOLDEN_DIR)
433         assert len(videos) == 6
434         cfg = ablation.litmus_config()
435         report = run_ablation(videos, cfg)
436         assert len(report.rows) == 1
437         row = report.rows[0]
438         assert row.signal == "gate_a"
439         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
440         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
441         assert row.sign_wins == 6 and row.sign_losses == 0
442         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
443         assert row.ci_low == pytest.approx(0.042, abs=0.01)
```

```
        assert row.ci_high == pytest.approx(0.104, abs=0.01)
        assert row.ci_excludes_zero is True
        # Gate-A is structural ? reported but never auto-promoted to earns_keep.
        assert row.verdict == ablation.STRUCTURAL_PROTECTED
        # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
        assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
        assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)


    @pytest.mark.skipif(not _golden_present(),
                        reason="golden output/*_golden_features.json absent")
    def test_litmus_report_is_well_formed():
        """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
        report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
        assert isinstance(report, AblationReport)
        assert report.num_videos == 6
        assert report.full_set_signals == ["gate_a"]
        assert report.label_set_hash
        out = ablation.format_table(report)
        assert "gate_a" in out


    # ------------------------------------------------------------------------------ edge
cases


    def test_min_players_gate_knob_zeroed():
        """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
        full = Variant(name="full", min_players=2, feature_names=["a"])
        ablated = ablation._ablated_variant(full, (), "min_players")
        assert ablated.min_players == 0
        assert ablated.feature_names == ["a"]            # columns untouched


    def test_ablated_label_falls_back_to_gate_knob():
        """With no columns, the ablated variant name carries the gate-knob label, not
'?'."""
        full = Variant(name="full", min_crossings=2)
        ablated = ablation._ablated_variant(full, (), "min_crossings")
        assert "min_crossings" in ablated.name


    # ------------------------------------------------------------------------------------
CLI


    def _write_learned_corpus(d: str, n: int = 6) -> None:
        for k in range(n):
            cands = []
            for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
((50.0, 51.0), 0)]:
                shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
                person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
                person["players_in_court"] = 4.0 if pos else 0.0
                person["in_court_ratio"] = 1.0 if pos else 0.0
                audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
                cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
                              "audio": audio, "label": pos})
            data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
                    "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
            with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
as f:
                json.dump(data, f)

```

```
503
504     def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
505         """``main()`` runs the LOGO ablation and writes both a PDF and the structured
JSON."""
506         pytest.importorskip("reportlab")
507         pytest.importorskip("pypdf")
508         d = str(tmp_path / "feat")
509         os.makedirs(d)
510         _write_learned_corpus(d)
511         pdf = str(tmp_path / "out.pdf")
512         js = str(tmp_path / "out.json")
513         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
"group",
514                                          "--pdf", pdf, "--json", js])
515         ablation.main()
516         out = capsys.readouterr().out
517         assert "Ablation report" in out
518         assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
519         assert os.path.isfile(js)
520         report = json.loads(open(js, encoding="utf-8").read())
521         assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
"audio"}
522
523
524     def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):
525         """``--litmus`` runs the static Gate-A config (single structural row)."""
526         d = str(tmp_path / "feat")
527         os.makedirs(d)
528         _write_learned_corpus(d)
529         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
530         ablation.main()
531         out = capsys.readouterr().out
532         assert "gate_a" in out
533         assert "static" in out                          # litmus full set is the static
gate variant
534
535
536     def test_cli_main_needs_two_videos(tmp_path, monkeypatch):
537         d = str(tmp_path / "feat")
538         os.makedirs(d)
539         _write_learned_corpus(d, n=1)
540         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d])
541         with pytest.raises(SystemExit):
542             ablation.main()
543
```

13. assistant (2026-06-18T01:11:19.241Z)

Now I have a clear picture. Let me verify a couple of key claims. First, the `_write_windowing_only` schema ? does it include `"label"` key like real lean files? Let me check whether there's a real windowing-only file to confirm the schema mirroring, and confirm `Sequence`/`Any`/`logger` are imported in ablation.py.

14. user (2026-06-18T01:11:21.793Z)

```
46:from __future__ import annotations
48:import argparse
49:import glob
50:import json
51:import logging
52:import os
53:from dataclasses import dataclass, field
54:from typing import Any, Dict, List, Optional, Sequence, Tuple
56:from backend.eval import experiment
57:from backend.eval.experiment import Variant, VideoCandidates
```

```
59:logger = logging.getLogger(__name__)
211:def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
248:def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
298:def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
318:def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) -> AblationReport:
386:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
453:    def _fmt_row(cells: Sequence[str]) -> str:
```

15. user (2026-06-18T01:11:23.441Z)

```
Badminton_BXH_2_golden_features.json
GX010128_golden_features.json
GX030094_golden_features.json
adarsh_avi_singles_golden_features.json
chunks
gbaaddy_golden_features.json
kushagra_singles_golden_features.json
largetest_doubles_golden_features.json
mahadevpura_1_golden_features.json
mahadevpura_2_golden_features.json
---real lean files search---
./output/Badminton_BXH_2_golden_features.json
./output/GX010128_golden_features.json
./output/GX030094_golden_features.json
./output/mahadevpura_1_golden_features.json
./output/mahadevpura_2_golden_features.json
```

16. assistant (2026-06-18T01:11:27.150Z)

Imports are all present. There are real golden files in `output/`. Let me inspect a real lean file and a real full-fusion file to verify the test schema mirrors reality and that the `_golden_present()` count would be exactly 6. This is critical for litmus faithfulness.

17. user (2026-06-18T01:11:33.686Z)

```
=== Total golden files ===
11

LEAN  n=  4  Badminton_BXH_2_golden_features.json  firstkeys=['active_frame_count',
'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start']
LEAN  n= 15  GX010128_golden_features.json  firstkeys=['active_frame_count', 'chunk_index',
'end', 'mean_velocity', 'peak_velocity', 'start']
LEAN  n= 30  GX030094_golden_features.json  firstkeys=['active_frame_count', 'chunk_index',
'end', 'mean_velocity', 'peak_velocity', 'start']
FULL  n=172  adarsh_avi_singles_golden_features.json  firstkeys=['audio', 'end', 'label',
'person', 'shuttle', 'start']
FULL  n= 82  gbaaddy_golden_features.json  firstkeys=['audio', 'end', 'label', 'person',
'shuttle', 'start']
FULL  n= 20  kushagra_singles_golden_features.json  firstkeys=['audio', 'end', 'label',
'person', 'shuttle', 'start']
FULL  n= 86  largetest_doubles_golden_features.json  firstkeys=['audio', 'end', 'label',
'person', 'shuttle', 'start']
LEAN  n=  1  mahadevpura_1_golden_features.json  firstkeys=['active_frame_count', 'chunk_index',
'end', 'mean_velocity', 'peak_velocity', 'start']
LEAN  n=  5  mahadevpura_2_golden_features.json  firstkeys=['active_frame_count', 'chunk_index',
'end', 'mean_velocity', 'peak_velocity', 'start']
FULL  n= 57  mahadevpura_singles_golden_features.json  firstkeys=['audio', 'end', 'label',
'person', 'shuttle', 'start']
FULL  n= 14  testlarge_short_golden_features.json  firstkeys=['audio', 'end', 'label', 'person',
'shuttle', 'start']
```

18. assistant (2026-06-18T01:11:38.532Z)

This is the real-world data. There are 11 files: exactly 6 FULL-fusion files and 5 LEAN files. The fix's `_golden_present()` now counts only the 6 FULL files, and `load_videos` loads exactly those 6. The real lean files' first-candidate keys are exactly `active_frame_count, chunk_index, end, mean_velocity, peak_velocity, start` ? which `_write_windowing_only` mirrors precisely (plus `track_id_count`).

Now let me verify the bug-pinning claim: would the new tests actually FAIL against the OLD load_videos? Let me check out the old version and run both new tests against it. Let me reconstruct the old `load_videos` and old `_golden_present`, then test.

19. user (2026-06-18T01:11:41.111Z)

```
Python 3.13.13
pytest 9.0.3
```

20. user (2026-06-18T01:12:09.959Z)

```
.............................                                            [100%]
29 passed in 24.32s
```

21. assistant (2026-06-18T01:12:14.804Z)

All 29 tests pass on the fixed code (and the litmus tests run, not skipped ? confirming the golden files are present and `_golden_present()` returns True with exactly 6). Now the critical adversarial check: do the two NEW tests actually FAIL against the OLD (pre-fix) `load_videos`? Let me temporarily reconstruct the old behavior in an isolated way and run the new tests against it.

22. user (2026-06-18T01:12:16.759Z)

```
got old ablation
got old test
```

23. assistant (2026-06-18T01:12:19.061Z)

I'll create a standalone script that imports the OLD `load_videos` and runs the two new tests' logic against it, to confirm they pin the bug. I'll do this in /tmp without touching the worktree.

24. user (2026-06-18T01:12:35.704Z)

```
OLD load_videos raised KeyError: 'shuttle'  -> new test_load_videos_skips_windowing_only_schema
WOULD FAIL on old code (bug pinned)
```

25. assistant (2026-06-18T01:12:42.280Z)

Confirmed: the OLD `load_videos` raises `KeyError: 'shuttle'` on the mixed corpus, so `test_load_videos_skips_windowing_only_schema` genuinely pins the bug (it would error out on old code at the `load_videos(d)` call). And `test_is_fusion_schema_distinguishes_lean_and_full` tests a function that doesn't exist in old code, so it would fail with AttributeError on old code ? also pins the new behavior.

Now let me verify the litmus numbers are actually reproduced on exactly the 6 FULL files (the core regression-faithfulness claim), and confirm the `test_load_videos_skips_windowing_only_schema` assertion `report.num_videos == 6` and single `gate_a` row are meaningful (not passing for wrong reason). The litmus tests already ran (29 passed, none skipped). Let me confirm they were not skipped.

26. user (2026-06-18T01:12:47.481Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
```

```
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-
fix
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 29 items / 24 deselected / 5 selected

tests/test_ablation.py::test_is_fusion_schema_distinguishes_lean_and_full PASSED [ 20%]
tests/test_ablation.py::test_load_videos_skips_windowing_only_schema PASSED [ 40%]
tests/test_ablation.py::test_litmus_reproduces_gate_a PASSED             [ 60%]
tests/test_ablation.py::test_litmus_report_is_well_formed PASSED         [ 80%]
tests/test_ablation.py::test_cli_main_litmus_flag PASSED                 [100%]

====================== 5 passed, 24 deselected in 2.07s ======================
```

27. assistant (2026-06-18T01:12:53.415Z)

The litmus tests run (not skipped) and pass ? meaning on exactly the 6 FULL golden files, `load_videos` returns 6 videos and reproduces ?F1?0.074, 6/6 wins, p?0.031. The fix is regression-faithful with the real mixed corpus present.

Now let me check a few adversarial edge cases the task flagged:

1. **A file with `person` but no `shuttle`** (partial schema) ? the predicate requires both. Let me verify behavior and whether it's tested.
2. Whether `load_videos` would crash on a FULL file that has `shuttle`/`person` keys on the first cand but a *later* cand missing them ? `_is_fusion_schema` uses `all(...)`, so it checks every cand. Good.
3. The `_write_windowing_only` lean schema ? real files have `track_id_count`? Let me check.

28. user (2026-06-18T01:12:58.146Z)

```
mahadevpura_1_golden_features.json -> ['active_frame_count', 'chunk_index', 'end',
'mean_velocity', 'peak_velocity', 'start', 'track_id_count']
GX010128_golden_features.json -> ['active_frame_count', 'chunk_index', 'end', 'mean_velocity',
'peak_velocity', 'start', 'track_id_count']
```

29. assistant (2026-06-18T01:13:07.691Z)

The real lean keys are exactly `active_frame_count, chunk_index, end, mean_velocity, peak_velocity, start, track_id_count` ? and `_write_windowing_only` produces precisely those 7 keys (test_ablation.py:365-366). The test fixture mirrors the real lean schema exactly.

Now let me consider the "partial schema" edge: a file with `shuttle` present but `person` missing (or vice versa). The predicate `all("shuttle" in c and "person" in c ...)` would return False, so the file is skipped rather than crashing on `c["person"]`. But there's a subtle case: what if a fusion file has `shuttle` and `person` but is missing one of the inner *columns* (e.g. `shuttle` dict lacks `duration`)? Then `c["shuttle"][fn]` at line 420 would still KeyError. Let me check whether that's a realistic concern and whether the predicate guards inner columns.

30. user (2026-06-18T01:13:11.402Z)

```
adarsh_avi_singles_golden_features.json missing_shuttle_cols= set() missing_person_cols= set()
kushagra_singles_golden_features.json missing_shuttle_cols= set() missing_person_cols= set()
testlarge_short_golden_features.json missing_shuttle_cols= set() missing_person_cols= set()
```

31. assistant (2026-06-18T01:13:15.819Z)

Real FULL files contain all inner columns, so the inner-column KeyError is not a present-day concern (the predicate guarding only the top-level `shuttle`/`person` presence is sufficient for the real corpus). This is a theoretical-only gap, not a must-fix.

Let me run the broader test suite quickly to make sure nothing else in the eval module is affected by the change, and check `ruff`/lint cleanliness on the changed files.

32. user (2026-06-18T01:13:17.721Z)

```
All checks passed!
--- ruff exit: 0 ---
```

33. assistant (2026-06-18T01:13:23.965Z)

Lint is clean. Let me do a final adversarial check on the assertions' meaningfulness. The task asks whether `report.num_videos == 6` and the single `gate_a` row could "pass for the wrong reason." Let me verify `num_videos` actually reflects loaded videos (not a hardcoded constant) and that `_write_learned_corpus` produces 6 distinct names `v0..v5` matching the assertion.

The assertion `{v.name for v in videos} == {f"v{k}" for k in range(6)}` is strong: it checks both count (6) AND the exact names, so a lean file leaking in (named e.g. `mahadevpura_1`) would break the set equality. That's a meaningful assertion that pins the skip behavior precisely.

One more: let me confirm `run_ablation`'s `num_videos` is derived from the input list length, so `== 6` is meaningful.

34. user (2026-06-18T01:13:25.233Z)

```
184:    num_videos: int
199:            "num_videos": self.num_videos,
340:        num_videos=len(videos),
447:              f"  ·  n={report.num_videos}  ·  IoU {report.iou}"
510:            f"contribution &mdash; n={report.num_videos} golden matches, IoU {report.iou}, "
```

35. assistant (2026-06-18T01:13:34.756Z)

`num_videos=len(videos)` (line 340) ? derived from the loaded list, so `report.num_videos == 6` genuinely confirms only 6 videos entered the ablation. The assertion is meaningful.

I have completed a thorough adversarial review. Let me summarize my verified findings.

Summary of what I verified:
- **Diff is correct and minimal**: `_is_fusion_schema` guard added to `load_videos`; `_golden_present()` now counts only full-fusion files.
- **Real corpus**: `output/` has exactly 11 files = 6 FULL + 5 LEAN. The fix loads exactly the 6 FULL files; litmus runs (not skipped) and reproduces ?F1?0.074, 6/6, p?0.031.
- **Bug is genuinely pinned**: reconstructed OLD `load_videos` raises `KeyError: 'shuttle'` on the mixed corpus; `_is_fusion_schema` doesn't exist in old code (AttributeError). Both new tests would fail on pre-fix code.
- **Lean fixture mirrors reality exactly**: `_write_windowing_only` produces the exact 7 keys of real lean files.
- **Assertions are meaningful**: set-equality on names (`{v0..v5}`), single `gate_a` row, `num_videos == len(videos) == 6`.
- **All 29 tests pass; ruff clean.**

Minor observations (not blocking): the predicate only guards top-level `shuttle`/`person` presence, not inner columns (line 420 `c["shuttle"][fn]` could still KeyError on a malformed-but-present `shuttle` dict ? not a concern for the current real corpus); and there's no explicit test for the asymmetric "person present, shuttle absent" partial case (covered implicitly by `all(... and ...)` but not pinned).

36. user (2026-06-18T01:13:53.985Z)

```
Structured output provided successfully
```

37. assistant (2026-06-18T01:14:02.402Z)

I completed the adversarial test-quality review. No must-fix found ? the change is safe to merge.

Key verified facts:

- The real `output/` corpus is exactly 11 files (6 full-fusion + 5 lean). The fix loads exactly the 6 ablatable files and the litmus tests RUN (not skipped) reproducing ?F1?0.074, 6/6 wins, p?0.031.
- Both new tests genuinely pin the bug: I reconstructed the pre-fix `load_videos` and confirmed it raises `KeyError: 'shuttle'` on the mixed corpus, and `_is_fusion_schema` doesn't exist pre-fix.
- `_write_windowing_only` mirrors the real lean schema exactly (7 keys verified against real `mahadevpura_1`/`GX010128` files).
- Assertions are meaningful (exact name-set `{v0..v5}`, single `gate_a` row, `num_videos == len(videos) == 6`).
- All 29 tests pass; ruff clean.

Two nits only (non-blocking): the predicate guards top-level `shuttle`/`person` presence but not inner columns (no concern for the current corpus), and there's no explicit test for the asymmetric "person present, shuttle absent" partial case.

Relevant files (absolute):
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix/backend/eval/ablation.py` (lines 386-428, 408-422 loader)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix/tests/test_ablation.py` (lines 360-399 new tests, 408-422 `_golden_present`)